CORE103L Computational Thinking
LW10 - Recursion I

DEPARTMENT OF COMPUTER SCIENCE

DHANANI SCHOOL OF SCIENCE AND ENGINEERING

HABIB UNIVERSITY

FALL 2026

Copyright @ 2026 Habib University

Contents

1 Introduction 2
1.1 Instructions . . . 2
1.2 Marking scheme . . . 2
1.2.1 Lab Objectives . . . 2
1.3 Late submission policy . . . 2
1.4 Use of AI . . . 3
1.5 Viva . . . 3

2 Prerequisites 4
2.1 Assert Statements . . . 4
2.2 Pytest . . . 4

3 Lab Exercises 6
3.1 Staircase . . . 6
3.2 Lucky Orders . . . 9
3.3 Treasure Hunt . . . 11

Introduction

1.1 Instructions

- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *LW10_aa123456.zip* where *aa123456* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise of both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 10.

1.2.1 Lab Objectives

Following are the lab objectives of this lab:

- Understand the fundamental concept of recursion in programming.
- Differentiate between recursive functions and iterative solutions.
- Explain the structure of recursive algorithms, including base cases and recursive calls.
- Write simple recursive functions to solve basic problems.

1.3 Late submission policy

There is no late submission policy for the lab.

1.4 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.5 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, "x should be a non-negative number"
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

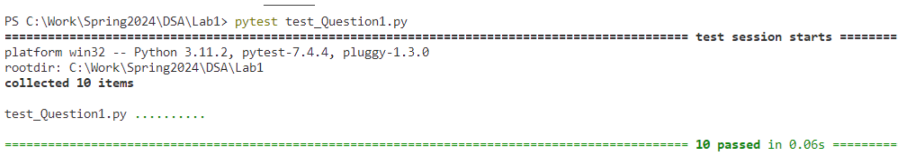
After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.

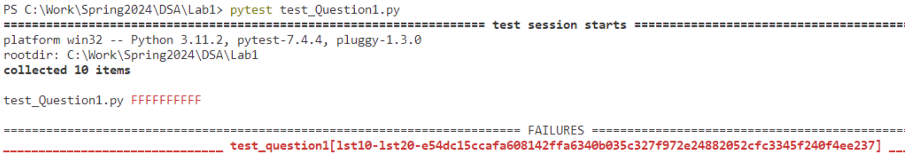


```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================================================================================== test session starts ========
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py ..........

============================================================================================== 10 passed in 0.06s =========
```

However, if all the test fail, then you should receive a similar output.



```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
==================================================================== test session starts ======================================
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py FFFFFFFFFF

========================================================================= FAILURES ============================================
_______________________________ test_question1[lst10-lst20-e54dc15ccafa608142ffa6340b035c327f972e24882052cfc3345f240f4ee237] __
```

Lab Exercises

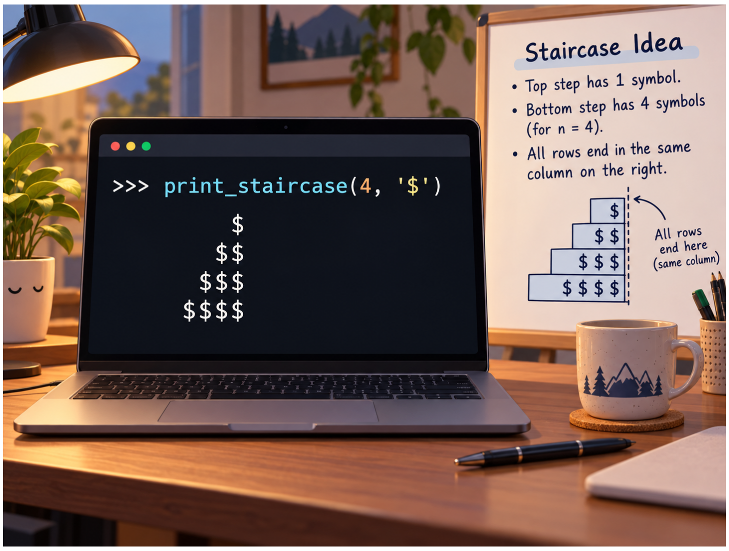
3.1 Staircase

This question should be implemented in the file ***q2.py****.*

Problem Description

The Computing Society is building a small terminal program for its Societies Week sign-up. To make the welcome screen more attractive, it prints a staircase using a chosen symbol, such as # or $.

The top step contains one symbol, and each step below contains one additional symbol. All steps are right-aligned, ending in the same column.



```
>>> print_staircase(4, '$')
   $
  $$
 $$$
$$$$
```

Staircase Idea

- Top step has 1 symbol.
- Bottom step has 4 symbols (for n = 4).
- All rows end in the same column on the right.

$
$ $
$ $ $
$ $ $ $
All rows end here (same column)

Function Description

Write a function **print_staircase** that takes two parameters, step_count and symbol, and prints a staircase using **recursion**.

The staircase has exactly step_count rows. The top row contains one symbol, and each subsequent row contains one additional symbol, ending with step_count symbols in the bottom row. Rows must be right-aligned using leading spaces, with no spaces between symbols. The function **prints** the staircase and returns nothing.

You must also define a **recursive helper function** that is called by **print_staircase**.

Prerequisite: Multiplying a String by an Integer

Multiplying a string by an integer repeats it.

```
>>> '*' * 3
'***'

>>> ' ' * 2 + '*' * 3
'  ***'
```

Prerequisite: Creating a Helper Function

A *helper function* is an additional function that can accept extra parameters to keep track of information during recursion. The required function starts the process by calling the helper, which performs the recursive calls.

Consider an unrelated example. The function **print_progress** takes total and prints steps from 1 to total, followed by **All done!**. The helper **print_steps** uses an additional parameter, current, to track the current step while keeping total unchanged.

```
def print_progress(total):
    print_steps(1, total)

def print_steps(current, total):
    if current > total:
        # Base case: stop recursion
        print('All done!')
    else:
        # Recursive case
        print('Step', current, 'of', total)
        print_steps(current + 1, total)
```

```
>>> print_progress(3)
Step 1 of 3
Step 2 of 3
Step 3 of 3
All done!
```

For the staircase problem, you must define your own recursive helper function. You may choose its name and parameters.

Sample Interaction

```
>>> print_staircase(1, '#')
#

>>> print_staircase(4, '$')
   $
```

```
  $$
 $$$
$$$$

>>> print_staircase(7, '@')
      @
     @@
    @@@
   @@@@
  @@@@@
 @@@@@@
@@@@@@@
```

Sample Interaction Explanation

For `print_staircase(1, '#')`, the staircase contains a single symbol with no leading spaces.

For `print_staircase(4, '$')`, the staircase has four steps:

Step	Leading spaces	Symbols	Printed line
1 (top)	3	1	`"   $"`
2	2	2	`"  $$"`
3	1	3	`" $$$"`
4 (bottom)	0	4	`"$$$$"`

Each step contains one more symbol and one fewer leading space than the step above it. The total number of leading spaces and symbols in each row is always step_count.

Constraints

- $1 \leq$ step_count ≤ 50.
- symbol is a single non-space character.
- You must use **recursion** through a helper function called by **print_staircase**.
- You may not use loops (**for** or **while**), comprehensions, or any other form of iteration.
- You may not use global variables.
- String concatenation and repetition are allowed.
- Print each row on a separate line without trailing spaces.

Testing

In order to test your function, type the following command on the terminal:

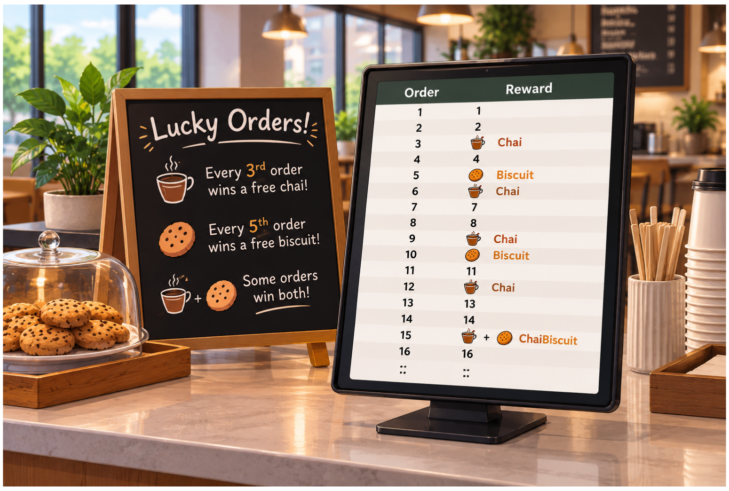
```
pytest tests/test_q4.py
```

3.2 Lucky Orders

*This question should be implemented in the file **q4.py**.*

Problem Description

Sky Dhaba is running a lucky-order promotion. Orders are numbered 1, 2, 3, and so on through the day. Every 3rd order (3, 6, 9, ...) wins a free chai, and every 5th order (5, 10, 15, ...) wins a free biscuit. An order that falls in both lists, such as 15, wins both. The cashier's screen goes through the order numbers from 1 to the last order of the day and shows what each customer wins.


Lucky Orders!
Every 3rd order wins a free chai!
Every 5th order wins a free biscuit!
Some orders win both!

Order	Reward
1	1
2	2
3	Chai
4	4
5	Biscuit
6	Chai
7	7
8	8
9	Chai
10	Biscuit
11	11
12	Chai
13	13
14	14
15	+ ChaiBiscuit
16	16
⁝	⁝

Function Description

Write a **recursive** function **print_rewards** that takes as a parameter last_order and prints one line for each order number from **1** to last_order, in increasing order:

- `Chai` if it is a multiple of 3 only,
- `Biscuit` if it is a multiple of 5 only,
- `ChaiBiscuit` if it is a multiple of both 3 and 5,
- the order number itself otherwise.

Constraints

- $1 \leq$ last_order ≤ 500.
- You must use **recursion**. You may not use loops or any other form of iteration, and you may not use global variables.
- You may not use a helper function.
- The function prints one item per line and returns nothing.

Sample Interaction

```
>>> print_rewards(1)
1

>>> print_rewards(5)
1
2
Chai
4
Biscuit

>>> print_rewards(16)
1
2
Chai
4
Biscuit
Chai
7
8
Chai
Biscuit
11
Chai
13
14
ChaiBiscuit
16
```

Sample Interaction Explanation

In the first sample, the only order is 1, which wins nothing, so its number is printed.

In the second sample, the last order is 5:

Order	Multiple of 3?	Multiple of 5?	Printed
1	no	no	`1`
2	no	no	`2`
3	yes	no	`Chai`
4	no	no	`4`
5	no	yes	`Biscuit`

In the third sample, order 15 is a multiple of both 3 and 5, so it wins `ChaiBiscuit`.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q5.py
```

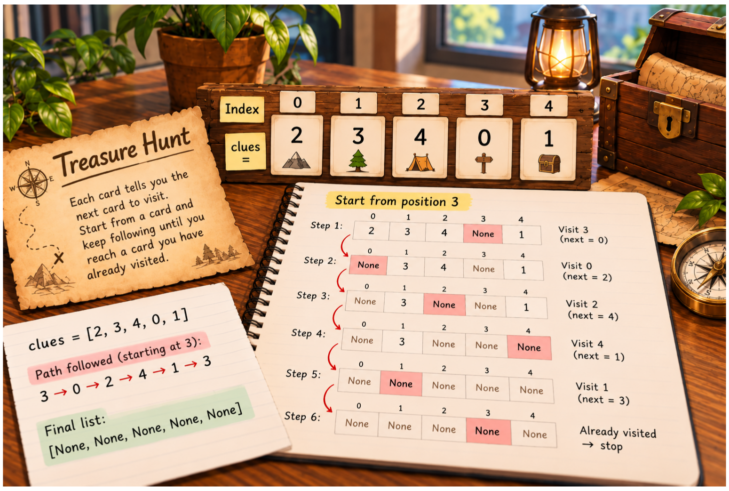
3.3 Treasure Hunt

This question should be implemented in the file ***q6.py****.*

Problem Description

During orientation week, your hall runs a scavenger hunt. Each clue card is numbered, and every card tells you the number of the next card to find. If there are n cards, they are numbered from **0** to **n - 1**, and the hunt is stored as a list: the value at position **i** is the number of the card to visit after card **i**.

You start at a given card, and keep following the cards until you reach one you have already visited. To remember which cards you have visited, replace their value in the list with **None**. Some cards may never be visited.


Treasure Hunt
N
E
S
W
Each card tells you the next card to visit. Start from a card and keep following until you reach a card you have already visited.
X
Index
0
1
2
3
4
clues =
2
3
4
0
1
clues = [2, 3, 4, 0, 1]
Path followed (starting at 3):
3 → 0 → 2 → 4 → 1 → 3
Final list:
[None, None, None, None, None]
Start from position 3
Step 1: 2 3 4 None 1
Visit 3 (next = 0)
Step 2: None 3 4 None 1
Visit 0 (next = 2)
Step 3: None 3 None None 1
Visit 2 (next = 4)
Step 4: None 3 None None None
Visit 4 (next = 1)
Step 5: None None None None None
Visit 1 (next = 3)
Step 6: None None None None None
Already visited → stop

Function Description

Write a **recursive** function **visit** that takes a list clues and an integer position.

It must visit the card at position, if it has not been visited, by reading its value and then setting it to **None**, and then visit the card that value points to, and so on.

The function modifies clues **in place**, and does not print or return anything.

Constraints

- $1 \leq$ **len(clues)** ≤ 100
- Every value in clues is an integer in **range(len(clues))**, and position is also in that range.
- clues contains no **None** values when the function is first called.

- You must use recursion. You may not use loops or any other form of iteration, and you may not use global variables.
- You may not use helper functions.
- The function must not return any value.

Sample Interaction

```
>>> clues = [2, 3, 4, 0, 1]
>>> visit(clues, 3)
>>> clues
[None, None, None, None, None]

>>> clues = [2, 0, 1, 4, 3]
>>> visit(clues, 4)
>>> clues
[2, 0, 1, None, None]

>>> clues = [0]
>>> visit(clues, 0)
>>> clues
[None]
```

Sample Interaction Explanation

In the first example, we start at position 3. At each step we mark the position as visited and follow the value we read from it:

Step	Visit position	Value read (next position)	`clues` afterwards
1	3	0	`[2, 3, 4, None, 1]`
2	0	2	`[None, 3, 4, None, 1]`
3	2	4	`[None, 3, None, None, 1]`
4	4	1	`[None, 3, None, None, None]`
5	1	3	`[None, None, None, None, None]`
6	3	already `None`	stop

In the second example, we start at position 4:

Step	Visit position	Value read (next position)	`clues` afterwards
1	4	3	`[2, 0, 1, 4, None]`
2	3	4	`[2, 0, 1, None, None]`
3	4	already `None`	stop

Positions 0, 1 and 2 are never reached, so they keep their values.

In the third example, the only card points to itself. It is marked once, and the second visit stops immediately.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q6.py
```